

Telerik Reporting

Integration plan & architecture — Contracts ERP

Backend: ASP.NET Core 8 / MediatR · Frontend: React 18 / Vite / KendoReact 16 · DevCraft subscription · 10 September 2026

Status: Draft for review · **Date:** 2026-09-10 · **Owner:** Emad Radwan

Context: KendoReact was upgraded to a DevCraft subscription (license at `~/telerik/telerik-license.txt`, valid to 2027-07-31). DevCraft includes **Telerik Reporting**, which we now want to adopt for in-app reports.

This document is architecture only — no code has been written. Read the **Recommendation**, **Risks**, and **Phased rollout** sections first.

1. Recommendation (short version)

Decision	Choice	Why
Hosting model	In-process Reporting REST service inside the existing API project	Single-box, single-container deployment on a resource-constrained ARM VM. Telerik Report Server is a separate product (own DB, own container, own user management, more RAM) and buys us nothing we need yet.
Report data binding	Reports pull JSON from dedicated <code>/api/report-data/*</code> endpoints backed by MediatR queries	Keeps all data access in Application-layer handlers (project convention: <i>no data access outside handlers, no repository layer</i>). Mirrors the existing <code>PaymentReportsController</code> pattern.
Report storage	<code>.trdp</code> files under <code>API/Reports/</code> , copied to output, resolved by a custom <code>IReportSourceResolver</code>	Version-controlled, reviewable in PRs, deployed with the app.
Frontend	Two tiers. Tier 1 (default): server renders to PDF/XLSX, frontend shows it with the existing <code>Blob → objectURL</code> pattern. Tier 2 (only where it earns its keep): embed <code>@progress/telerik-react-report-viewer</code> on specific screens.	Avoids dragging the jQuery-based HTML5 viewer + its script loader into every report. Most ERP reports just need "give me the PDF".
Report authoring	Web Report Designer hosted in a small internal tool/route — <i>not</i> the Standalone Designer	The Standalone / VS Report Designer is Windows-only (WPF) . Primary dev machine is macOS. See §7.
QuestPDF	Keep for now, converge later — pending the SkiaSharp conflict check in §9.1	Only one report uses QuestPDF today (the Arabic payment voucher). No reason to rush a migration; every reason to first prove Telerik works in our container.

2. Current state (what's in the repo today)

Backend — .NET 8, Clean Architecture: `API` → `Application` / `Infrastructure` → `Domain` / `Persistence` .

- CQRS via MediatR, nested-class feature files, handlers return `Result<T>` .
- `DataContext` injected directly into handlers; **no repository layer**; lazy loading disabled.
- Auth: JWT bearer. OData list endpoints. SignalR hubs. Serilog (Warning min).
- SPA served as static files from `API/wwwroot` + `app.MapFallbackToFile("index.html")` .

PDF / reporting today

- `Infrastructure/Pdf/PdfGenerationService.cs` (`IPdfGenerationService`) — **QuestPDF**, ~442 lines, used in exactly **one** place: `PaymentReportsController.GetPaymentReportPdf` (Arabic payment voucher).
- `QuestPDF.Settings.UseEnvironmentFonts = false` and `CheckIfAllTextGlyphsAreAvailable = false` set globally in `Program.cs` ; fonts bundled in `API/wwwroot/fonts` and force-copied on publish.
- Everything else is either **Power BI** (external) or **client-side Excel** generation via `@progress/kendo-ooxml` (dozens of `*Excel.tsx` components), or an ad-hoc **HTML** → **headless-Chrome** → **PDF** pipeline for one-off deliverables.
- Frontend PDF consumption pattern (already established, `EditPaymentForm.tsx`): RTK Query fetches `ArrayBuffer` → `new Blob([...], {type:"application/pdf"})` → `URL.createObjectURL` → open / download.

Frontend — React 18.3.1, Vite 5, **KendoReact 16.0.0**, Yarn 4. jQuery is **not** currently a dependency.

Deployment (see the deploy runbook / memory)

- Single Oracle Cloud ARM box (`business2` , VM.Standard.A1.Flex, 2 OCPU / 12 GB, **4 GB swap**).
- One container from `Dockerfile.vm` : build on `mcr.microsoft.com/dotnet/sdk:8.0` (linux/arm64), runtime on `mcr.microsoft.com/dotnet/aspnet:8.0.20-bookworm-slim` .
- **On-box dotnet publish is already memory-fragile** — Roslyn needs >6 GB for `Persistence` ; builds are pinned to one core (`--cpuset-cpus 0`), no `--memory` cap, run inside `tmux` .
- **No CI**. The SPA is built locally and committed to `API/wwwroot` ; the image is built on the VM.
- `Dockerfile.vm` currently installs **no fonts** and **no fontconfig** — QuestPDF works only because it ships its own font and SkiaSharp native assets.

3. Licensing & package acquisition

Telerik Reporting is covered by the same DevCraft subscription as KendoReact, but it is delivered through the **Telerik private NuGet feed** (and an npm package for the viewer), not nuget.org.

3.1 NuGet feed access

- Feed: `https://nuget.telerik.com/v3/index.json`
- Auth: username `api-key` , password = the license/API key. Add a repo-root `nuget.config` :

```
``xml <?xml version="1.0" encoding="utf-8"?> <configuration> <packageSources> <add key="nuget.org" value="https://api.nuget.org/v3/index.json" protocolVersion="3" /> <add
```

```
key="telerik" value="https://nuget.telerik.com/v3/index.json" /> </packageSources>
<packageSourceCredentials> <telerik> <add key="Username" value="api-key" /> <add
key="ClearTextPassword" value="%TELERIK_NUGET_KEY%" /> </telerik> </packageSourceCredentials>
<packageSourceMapping> <packageSource key="nuget.org"><package pattern="" /></packageSource>
<packageSource key="telerik"><package pattern="Telerik." /></packageSource>
</packageSourceMapping> </configuration> ``
```

`nuget.config` is committed **without** the key; the key comes from the `TELERIK_NUGET_KEY` environment variable (local shell + Docker build arg). Add `packageSourceMapping` so a typo can never pull a `Telerik.*` package from a public source.

3.2 Runtime license (no watermark)

- Add the **Telerik.Licensing** NuGet package (same mechanism as KendoReact's licensing). It reads, in order: `TELERIK_LICENSE` env var (file contents) → `TELERIK_LICENSE_PATH` → `~/.telerik/telerik-license.txt`.
- **Locally:** already works — the file is at `~/.telerik/telerik-license.txt`.
- **In Docker:** the home directory is `/root` and the file is not in the image. Pass the license as a build/runtime value:
- Build stage: `ARG TELERIK_LICENSE` → `ENV TELERIK_LICENSE=$TELERIK_LICENSE` so the build-time license check passes and no watermark is baked in.
- Runtime: also set `TELERIK_LICENSE` in `docker-compose.vm.yml` `environment:` (from a `.env` value, like `TokenKey` / `SendGridKey` are handled today).
- Without this, **every rendered report gets a "Trial Version" banner** — exactly the KendoReact watermark situation, same fix.

3.3 Version pinning

- Pin **one exact** Telerik Reporting version (the current release line is 19.x for .NET 8 — confirm the latest at install time) and use it **verbatim** for:
- `Telerik.Reporting`, `Telerik.Reporting.Services.AspNetCore`, `Telerik.Reporting.OpenXmlRendering`, etc.
- `@progress/telerik-react-report-viewer` and `@progress/telerik-web-report-designer` on the frontend.
- The JS viewer/designer version **must equal** the REST service version or the viewer throws version-mismatch errors. Bump them together, always.

4. Hosting model — in-process REST service

Add to **API** :

```
Telerik.Reporting
Telerik.Reporting.Services.AspNetCore
Telerik.Reporting.OpenXmlRendering      (XLSX / DOCX / PPTX output)
Telerik.Licensing
```

Wire-up in `Program.cs` (respecting the existing pipeline order — CORS → AuthN → Routing → AuthZ):

- `builder.Services.AddControllers().AddNewtonsoftJson()` — **note:** the Reporting REST service historically requires `Newtonsoft.Json` on the MVC pipeline. We currently use `System.Text.Json` implicitly. This needs verification against the pinned version; if still required, scope it carefully so it doesn't change serialization for existing controllers (our Domain entities use a Newtonsoft `SnakeCaseNamingStrategy` attribute already, so Newtonsoft is not foreign to the codebase, but the API DTOs assume STJ/PascalCase).
- Register a `ReportServiceConfiguration` singleton:
- `ReportingEngineConfiguration` = built from `IConfiguration`
- `HostAppId` = `"ContractsErp"`
- `Storage` = `FileStorage` (cache dir under the container's writable `/app/logs` -style volume, or `/tmp`)
- `ReportSourceResolver` = **custom** (see §5 and §6)
- Map the controller: a thin `ReportsController : ReportsControllerBase` at `~/api/reports`.
- **Auth:** the REST service endpoints must sit behind the same JWT policy as the rest of `/api`. The React viewer sends a bearer token via its `serviceUrl` + `getReport` auth hook; confirm our `Accept-Language` / `GetLanguage()` story still works (the viewer can pass headers).

CORS: in Development the SPA is on `:3000` and API on `:5001` — the existing `CorsPolicy` already covers this; just make sure the Reporting endpoints are not excluded.

5. Report data binding (the important architectural decision)

Telerik reports can bind to `SqlDataSource`, `ObjectDataSource`, `JsonDataSource`, `WebServiceDataSource`, `CsvDataSource`.

Do not use `SqlDataSource`. It would put raw SQL and connection strings inside `.trdp` files, bypassing the Application layer, EF, multilingual label resolution, and every business rule in the handlers. That breaks the core project convention.

Recommended: `WebServiceDataSource` (or `JsonDataSource` over a URL) → dedicated report-data endpoints.

```
API/Controllers/Report/ReportData/
  PaymentVoucherReportDataController.cs → GET /api/report-data/payment-voucher/{id}
  TrialBalanceReportDataController.cs    → GET /api/report-data/trial-balance?...
  ...
```

- Each endpoint is a `BaseApiController` action that does `Mediator.Send(new GetXForReport.Query(...))` and returns a flat JSON DTO shaped for the report (PascalCase, standard `EntityDto` convention).
- New CQRS files follow the existing name: `GetPaymentForReport.cs` already exists and is the template.
- The report's `WebServiceDataSource.Url` points at `{applicationUrl}/api/report-data/...`; parameters map to route/query values; the bearer token is forwarded by the REST service.
- Benefits: report DTOs are testable (`ApplicationTests`), data logic stays in one place, `Accept-Language` continues to drive labels, and the report designer just sees JSON fields.

Alternative considered — `ObjectDataSource` to a class in a `Reporting` class library that calls `IMediator`: rejected. It couples report DLLs to `Application`, complicates the designer (needs the assembly

+ a DI container), and gives no test or caching benefit over the endpoint approach.

6. Report storage & source control

```
API/Reports/  
  PaymentVoucher.trdp  
  TrialBalance.trdp  
  _shared/StyleSheet.trss      (shared theme: fonts, colors, RTL defaults)  
  README.md
```

- `.trdp` (packaged binary) is the practical choice; `.trdx` (XML) diffs slightly better but tooling support is weaker. Pick one and be consistent — `.trdp`.
- `.csproj`: `<Content Include="Reports\**\*.trdp">`
`<CopyToOutputDirectory>PreserveNewest</CopyToOutputDirectory></Content>` (same mechanism already used for `wwwroot/fonts` and the logo).
- Custom `IReportSourceResolver`: maps an incoming report **key** (e.g. `"PaymentVoucher"`) to `Path.Combine(env.ContentRootPath, "Reports", key + ".trdp")`. Never accept a raw file path from the client (path traversal). Whitelist known keys or sanitize hard.
- `Dockerfile.vm`: `COPY --from=build-env /src/API/Reports ./Reports` (like the `Json` folder).

7. Report authoring — the macOS constraint

Problem: the rich designers are Windows-only:

- **Standalone Report Designer** — WPF desktop app, Windows only.
- **VS Report Designer** — Visual Studio (Windows) only. Rider/VS-for-Mac: not supported.

Options, best first:

1. **Host the Web Report Designer** (`@progress/telerik-web-report-designer` , Angular-based, embeddable) behind an admin-only route in our SPA or a tiny separate internal page. It talks to the same REST service, reads/writes `.trdp` in `API/Reports/`. This is the only fully macOS-native path and keeps report edits reviewable (the files still land in git).
2. Cost: it's a chunky component with its own toolbar/CSS; needs a design-surface REST endpoint (`DesignerController`) and a writable report directory in **dev** (not prod — prod reports are immutable, shipped in the image).
3. **Windows VM** (Parallels / UTM / a cloud Windows box) running the Standalone Designer, editing files on a shared folder. Fastest designer UX, no code, but a clunky workflow and an extra license seat consideration.
4. **Hand-author `.trdx` XML** — viable for simple tabular reports, painful for anything with grouping/charts. Not recommended as the primary workflow.

Recommendation: stand up the **Web Report Designer in dev** (option 1). Treat `API/Reports/*.trdp` as source; production never edits them.

8. Frontend integration

Tier 1 — server-rendered document (default for most reports)

No viewer, no jQuery. A report-data-independent endpoint renders the report to a document:

```
POST /api/reports/render
{ report: "PaymentVoucher", format: "PDF", parameters: { paymentId: "..." } }
→ 200 application/pdf (bytes)
```

Implement with `ReportProcessor.RenderReport("PDF", reportSource, deviceInfo)` inside a normal `BaseApiController` action (or a `Result<byte[]>` handler + `File(...)`). Frontend reuses the **exact** pattern already in `EditPaymentForm.tsx` / `paymentsApi.ts`: RTK Query `query<ArrayBuffer,...>` → `Blob` → `objectURL` → open/download.

Formats available: `PDF`, `XLSX`, `DOCX`, `CSV`, `IMAGE`, `PPTX` (needs `OpenXmlRendering` package).

This covers: payment vouchers, trial balance, GL statements, project reports, commission sheets — i.e. "print / export this".

Tier 2 — interactive React viewer (only where it pays off)

`@progress/telerik-react-report-viewer` for screens that genuinely need: parameter panels the user tweaks live, drill-through / interactive sorting, in-browser print preview, page navigation.

Friction to budget for:

- The React wrapper wraps the **HTML5 viewer**, which **depends on jQuery** and on Telerik's `telerikReportViewer-*.js` being loadable. In a Vite app this means either adding `jquery` as a dep and importing the Telerik viewer scripts, or loading them via `<script>` tags in `index.html`. This is the single biggest reason to keep Tier 2 rare.
- Version lock with the REST service (§3.3).
- Theming/CSS to match the KendoReact 16 look.

Plan: ship Tier 1 first. Add the React viewer on **one** pilot screen once Tier 1 is proven, evaluate the jQuery/bundle cost, then decide whether to expand.

9. Docker / deployment changes

9.1 ⚠ SkiaSharp version conflict — verify before anything else

- `Infrastructure.csproj` pins `SkiaSharp.NativeAssets.Linux 3.119.1` (for QuestPDF 2025.12).
- Telerik Reporting renders graphics/text on Linux through **its own SkiaSharp dependency**, which on recent-but-not-latest Reporting releases is **SkiaSharp 2.88.x**.
- If Telerik Reporting requires SkiaSharp 2.88.x and QuestPDF requires 3.x in the **same process**, that's a hard diamond conflict — NuGet will pick one and the other engine may crash at render time (native ABI mismatch, not just a warning).

Action item (do this first, it gates the whole plan):

1. In a scratch `net8.0` project, add the pinned `Telerik.Reporting` + `Telerik.Reporting.Services.AspNetCore` and run `dotnet list package --include-transitive | grep -i skia`.
2. Compare with QuestPDF 2025.12's SkiaSharp requirement.
3. If they diverge across a major version → **converge on one PDF engine**. Given QuestPDF has exactly one consumer, the likely outcome is: **port the payment voucher to Telerik and remove QuestPDF**. Budget for that; it's ~1 report but it's an Arabic RTL pixel-perfect one.

9.2 Fonts (required — currently absent)

`Dockerfile.vm` runtime stage needs fonts + fontconfig for Telerik Reporting, especially Arabic:

```
RUN apt-get update && apt-get install -y --no-install-recommends \
    libfontconfig1 libfreetype6 fontconfig \
    fonts-noto-core fonts-noto-cjk \
    && rm -rf /var/lib/apt/lists/*
# Plus our bundled Arabic faces (Amiri / Cairo / Noto Naskh Arabic) copied into
# /usr/share/fonts/truetype/erp/ and `fc-cache -f`.
```

- Reuse the faces already in `API/wwwroot/fonts` where possible; add a proper Arabic naskh face if the voucher needs one.
- Telerik reports reference fonts **by family name** — the report's font names must exactly match installed family names. Encode this in the shared `.trss` stylesheet (§6).

9.3 SkiaSharp native assets on ARM64

- Confirm the resolved SkiaSharp version publishes `linux-arm64` native assets (2.88.6+ and 3.x both do, but verify the exact one). `SkiaSharp.NativeAssets.Linux.NoDependencies` + the `apt` libs above is the usual combination.
- The image is `--platform=linux/arm64` throughout, so `dotnet publish` must restore the arm64 RID assets.

9.4 Build memory

- Telerik Reporting pulls in a sizeable assembly set. On-box `dotnet publish` is **already** the thing that has wedged the VM three times (Roslyn >6 GB, only 4 GB swap).
- Mitigations: keep the one-core pin, consider bumping swap to 6–8 GB before the first Telerik build, and strongly consider **building the image off-box** (locally on the Mac, `docker buildx` for arm64, push to a registry) so the production VM only ever `docker pull`s. This is worth doing regardless of Telerik and would de-risk every future deploy.

9.5 Writable cache directory

- The Reporting REST service and designer need a writable temp/cache dir. The container already mounts `/home/ubuntu/logs/contracts:/app/logs`. Add a dedicated `reports-cache` volume or point the storage at `/tmp` (ephemeral is fine for the render cache).
-

10. Arabic / RTL

- Reports are bilingual (the voucher filename is literally `..._بيان_دفعة...pdf`).
- Telerik Reporting supports RTL: set `Report.RightToLeft`, per-textbox `RightToLeft`, and use a font with full Arabic coverage + shaping (Skia + HarfBuzz handles shaping; `HarfBuzzSharp.NativeAssets.Linux` is already referenced).
- Number/date formatting: drive from the `Accept-Language` header via the report-data endpoint (format server-side into strings where locale-sensitive), or pass a `culture` report parameter.
- Test the voucher early — it's the hardest case and the current QuestPDF benchmark to match.

11. Risks & open questions

#	Risk / question	Impact	Mitigation
R1	SkiaSharp 2.88 vs 3.x conflict with QuestPDF (§9.1)	High — could force removing QuestPDF	Spike first; plan to converge on Telerik
R2	Reporting REST service still mandates Newtonsoft.Json on MVC	Medium — serialization behavior change risk for existing API DTOs	Verify vs pinned version; scope Newtonsoft to reporting controllers only if possible
R3	No macOS designer (§7)	Medium — workflow friction	Host Web Report Designer in dev
R4	React viewer's jQuery / script-loader baggage in Vite	Medium — bundle size, setup pain	Default to Tier 1 (server-rendered); Tier 2 only where needed
R5	Fonts absent in the container image	High — garbled / missing glyphs, esp. Arabic	Add fontconfig + faces to <code>Dockerfile.vm</code> (§9.2)
R6	On-box build memory (§9.4)	High — repeat of the VM lockout	Bump swap; move image builds off-box
R7	License watermark in Docker (§3.2)	High — unusable output	Pass <code>TELERIK_LICENSE</code> as build arg + runtime env
R8	Viewer/service version drift	Medium — runtime errors	Pin both, bump together, document in this file
R9	ARM64 SkiaSharp native assets	Medium	Verify RID assets resolve for <code>linux-arm64</code>
R10	Report-data endpoints add surface area / auth review	Low	Same <code>BaseApiController</code> + JWT policy; treat like any <code>/api</code> route

12. Phased rollout

Phase 0 — De-risk (½–1 day, no committed code)

- Scratch project: resolve Telerik Reporting's transitive **SkiaSharp** version; compare to QuestPDF (§9.1).

- Confirm the current Reporting release, .NET 8 support, Newtonsoft requirement, `linux-arm64` assets.
- Confirm the DevCraft subscription entitles Telerik Reporting + Web Report Designer (check the account portal).
- Decision checkpoint: **coexist with QuestPDF** or **converge on Telerik**.

Phase 1 — Backend skeleton

- `nuget.config` with Telerik feed + `packageSourceMapping` (key via `TELERIK_NUGET_KEY`).
- Add packages to `API`: `Telerik.Reporting`, `.Services.AspNetCore`, `.OpenXmlRendering`, `Telerik.Licensing`.
- `Program.cs`: register `ReportServiceConfiguration`, custom `IReportSourceResolver`, map `ReportsController` at `/api/reports` behind JWT.
- `API/Reports/` folder + `.csproj` content copy + shared `.trss`.
- `POST /api/reports/render` Tier-1 endpoint (`ReportProcessor.RenderReport`).
- **Stop and report — no build** (per project build rules).

Phase 2 — First vertical slice: Payment Voucher

- `GetPaymentForReport` already exists → add `PaymentVoucherReportDataController` (`/api/report-data/payment-voucher/{id}`).
- Author `PaymentVoucher.trdp` (Web Designer) bound via `WebServiceDataSource`, RTL, Arabic font.
- Frontend: new RTK Query endpoint + reuse `EditPaymentForm` Blob pattern; add a "Print (Telerik)" action next to the existing one for A/B comparison.
- Visual QA against the current QuestPDF output.

Phase 3 — Docker

- `Dockerfile.vm`: fonts + fontconfig, `COPY` the `Reports` folder, `ARG/ENV TELERIK_LICENSE`, `ARG TELERIK_NUGET_KEY` for restore.
- `docker-compose.vm.yml`: `TELERIK_LICENSE` in `environment`, `reports-cache` volume.
- Bump VM swap to 6–8 GB **or** switch to off-box `docker buildx` + registry pull.
- Test render in a locally-built arm64 container before touching the VM.

Phase 4 — Web Report Designer (dev only)

- `DesignerController` + writable `Reports` dir in Development.
- Admin-only route hosting `@progress/telerik-web-report-designer`.
- Document the "edit → save → commit `.trdp` → PR" workflow.

Phase 5 — Expand + decide on Tier 2

- Port 2–3 more reports (trial balance, a project report, a commission sheet).
- Pilot the React viewer on one parameter-heavy screen; measure bundle impact; decide go/no-go on wider use.
- If converging: port the voucher fully, delete `PdfGenerationService` + QuestPDF, drop `SkiaSharp` pin if unused.

13. Out of scope (for now)

- Telerik **Report Server** (scheduling, subscriptions, a report catalog UI). Revisit only if we need scheduled email delivery of reports or non-developer report management.
 - Replacing the client-side `kendo-ooxml` Excel exports — those work and are cheap; leave them.
 - Power BI — unaffected; it stays the analytics layer.
-

14. Decisions needed from Emad

1. **Phase 0 outcome pending**, but directionally: coexist with QuestPDF, or commit to converging on Telerik?
2. Is an **off-box image build** (Mac `buildx` → registry → VM pull) acceptable? It de-risks R6 permanently.
3. Web Report Designer in dev — OK to add an admin route in the main SPA, or prefer a separate tiny internal app?
4. First report to target — Payment Voucher (proposed, since `GetPaymentForReport` exists), or something else?